2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / 1.4.1(h) LESSON 006 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 006 · Floating-point addition

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Convert 01011000 0011 to denary. Use an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

Q2. Represent 0.375 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

Q3. Convert 11110110 from 8-bit two’s-complement binary to denary. Show signed place values.

Q4. Convert AD_{16} to binary, then logically shift the 8-bit value right once. Give the final denary value.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — Exponent alignment

Recall the term — Arithmetic right shift

Recall the term — Sign extension

Recall the term — Guard bits

Recall the term — Renormalisation

Recall the term — Rounding error

New learning

Align the exponents — Choose the larger exponent. Increase the smaller exponent one step at a time and right-shift that mantissa once for each step.

Preserve sign and working precision — An arithmetic right shift repeats 0 for a positive mantissa and 1 for a negative mantissa. — Keep low-order guard bits when a shift would otherwise discard a non-zero bit.

Add using a wider working sum — Sign-extend both aligned mantissas before adding so the calculation cannot wrap into the wrong sign. — CODE:
requested mantissa: 6 bits
working sum: 7 bits

Normalise and check — If the sum is too large for the mantissa, shift right and increase the exponent. If redundant sign bits remain, shift left and decrease the exponent.

Worked example

Add 5 + 1.5

1. Use the common exponent 3 — Raise the second exponent from 1 to 3, so shift its mantissa right twice using sign extension. — CODE:
010100 0011
011000 0001 → 000110 0011

2. Use a temporary extra sign bit — CODE:
0010100
+ 0000110
= 0011010 — The working sum is 26/32 × 2^3 .

3. Return to the requested width — CODE:
0011010

Do Now Recall previous learning | I Do Read new knowledge | We Do Practise with guidance | You Do Work independently | Review & Improve Check my understanding

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

The unnormalised sum is 32/32 × 2 3 . Normalise and return to the requested field widths.
CODE:
Final: 010000 0100
Denary check: 6 + (2) = 8.
Common exponent 3 (1), second mantissa 001000 (1), widened sum 0100000 (1), final mantissa 010000 (1), exponent 0100 (1).

Q3. Use common exponent 3 (0011). Shift the smaller-scale mantissa right, repeating its sign bit.
CODE:
Aligned mantissas: 101100 and 000110
Temporary extra sign bit, same fractional places:
1101100
+ 0000110
= 1110010
The unnormalised sum is -14/32 × 2 3 . Normalise and return to the requested field widths.
CODE:
Final: 100100 0010
Denary check: -5 + (1.5) = -3.5.
Common exponent 3 (1), second mantissa 000110 (1), signed sum −14/32 (1), normalised mantissa 100100 (1), exponent 0010 (1).

Q4. a) Matching positions must have the same scaled weight (1), so operands need the same exponent before direct addition (1). b) Repeat the sign bit / insert 1 (1), preserving the negative sign as the mantissa is halved (1). c) A short mantissa may discard nonzero low-order bits during alignment or final storage (1).

Exit Ticket · Model answer

True. Align the exponents first so the mantissas represent values at the same scale.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

Do Now Recall previous learning | I Do Read new knowledge | We Do Practise with guidance | You Do Work independently | Review & Improve Check my understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. 5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$.

Q2. 01100000 1111 0.375 = $0.011_2 = 0.11_2 \times 2^{-1}$. Mantissa 01100000 ; exponent 1111 = −1 .

Q3. −10 −128 + 64 + 32 + 16 + 4 + 2 = −10 .

Q4. $AD_{16} = 10101101_2 \rightarrow 01010110_2 = 86$ A logical right shift inserts 0 on the left and discards the rightmost bit.

You Do pages · Independent questions

Q1. Use common exponent 6 (0110). Shift the smaller-scale mantissa right, repeating its sign bit.
CODE:
Aligned mantissas: 011000 and 000101
Temporary extra sign bit, same fractional places:
0011000
+ 0000101
= 0011101
The unnormalised sum is 29/32 × 2 6 . Normalise and return to the requested field widths.
CODE:
Final: 011101 0110
Denary check: 48 + (10) = 58.
Common exponent 6 (1), second mantissa 000101 (1), correct addition (1), final mantissa 011101 (1), exponent 0110 (1).

Q2. Use common exponent 3 (0011). Shift the smaller-scale mantissa right, repeating its sign bit.
CODE:
Aligned mantissas: 011000 and 001000
Temporary extra sign bit, same fractional places:
0011000
+ 0001000
= 0100000

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

→ 011010 0011 — The mantissa already begins 01 , so the exponent stays at 3.

4. Verify in denary — CODE:
5 + 1.5 = 6.5

We Do · Practise with guidance

2. Rescale the sum: 6 + 5

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

Build the answer

3. A negative operand: −5 + 3

4. Align a negative mantissa: −3 + (−2)

5. Precision loss

Use a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. State whether an exact normalised floating-point result can be stored.

Use the rounding or truncation rule specified by the question.

OCR SAMPLE ASSESSMENT · Q7(e) · [3]

Use a four-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Give the final answer in normalised floating-point form.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Question 1 · OCR OCTOBER 2021 Q2(c) · [5]
Add 011000 0110 and 010100 0100 . Show your working and give your answer in normalised floating-point form using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent.
Paraphrased exam question; original operands and marks retained. OCR October 2021, Q2(c) .

Question 2

Question 2 · ADAPTED EXAM PRACTICE · [5]
Add 011000 0011 and 010000 0010 using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show alignment and the widened sum, then give the answer in normalised floating-point form.
Adapted from the OCR addition task: changed values; classroom marks.

Question 3

Question 3 · ORIGINAL EXAM-STYLE EXTENSION · [5]
Add 101100 0011 and 011000 0001 using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent. Show alignment and signed addition, then give the answer in normalised floating-point form.

Question 4

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Question 4 · ORIGINAL EXAM-STYLE EXPLANATIONS · [5]
Why must exponents match before mantissas are added directly? [2]
What enters from the left when a negative mantissa is shifted right for alignment, and why? [2]
Why can addition lose precision even if the result is within exponent range? [1]

Exit Ticket · Answer before checking

Floating-point mantissas must be aligned to the same exponent before they are added directly.